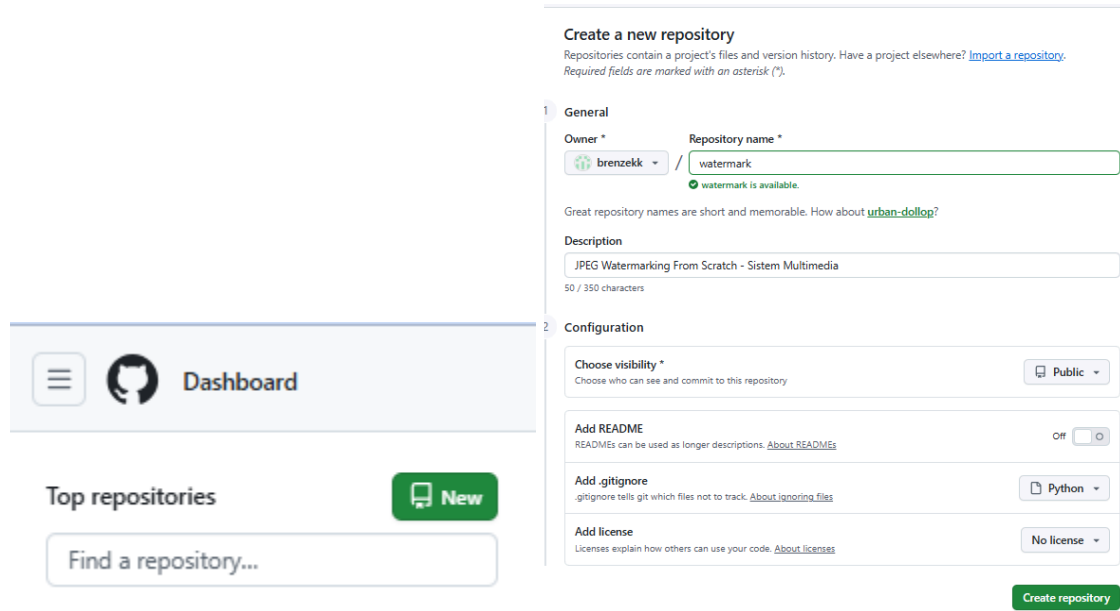


Watermark Wajah

1. Membuat repositori dan Strukturnya



Create a new repository
Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk (*).

1 General

Owner * brenzekk / Repository name * watermark
watermark is available.

Great repository names are short and memorable. How about [urban-dollop](#)?

Description
JPEG Watermarking From Scratch - Sistem Multimedia
50 / 350 characters

2 Configuration

Choose visibility * Public
Choose who can see and commit to this repository

Add README
READMEs can be used as longer descriptions. [About READMEs](#) Off

Add .gitignore
.gitignore tells git which files not to track. [About ignoring files](#) Python

Add license
Licenses explain how others can use your code. [About licenses](#) No license

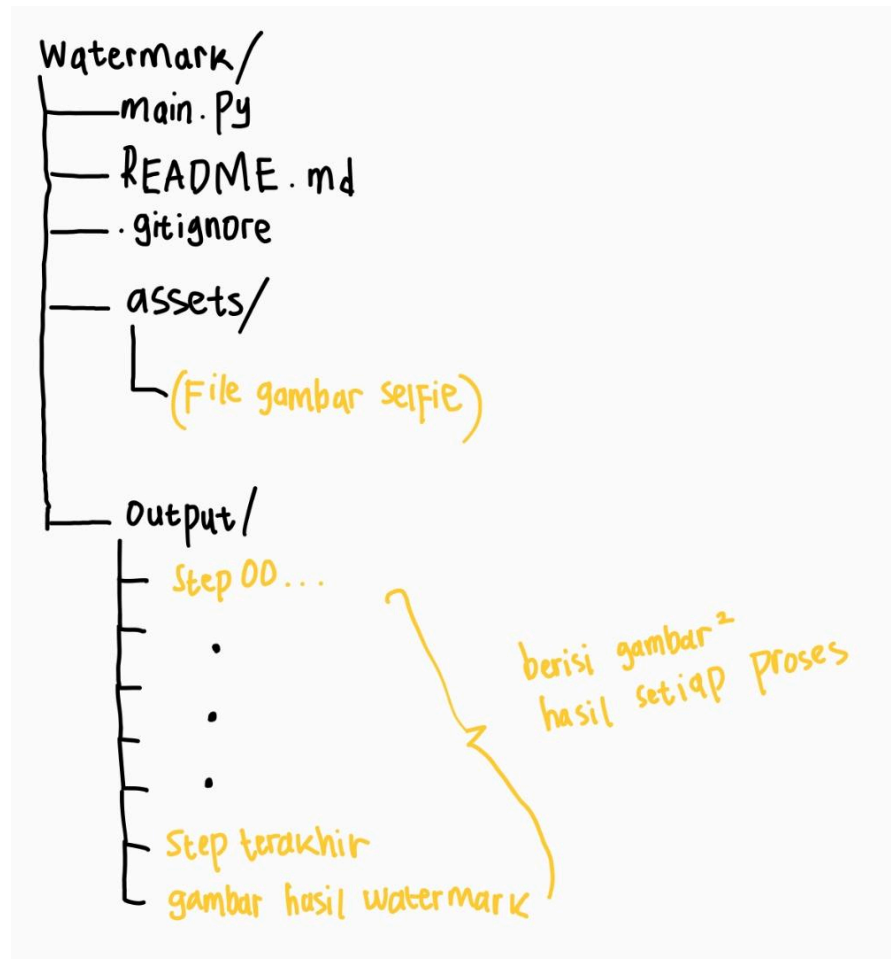
[Create repository](#)

Klik “new”, lalu isi menyesuaikan dengan judul yang diinginkan, “public”, dan terakhir “create repository”.

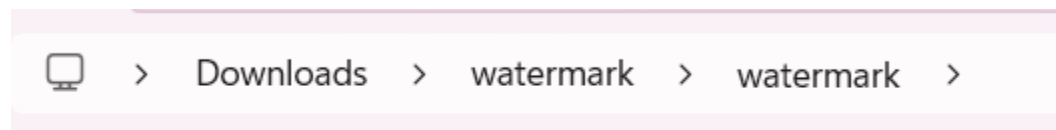


watermark

Setelah klik “create repository” akan diarahkan filenya ingin disimpan di mana? Saya menyimpannya ke bagian Download dengan nama watermark. Folder sebagai wadah repository sudah terbuat lalu saatnya mengisi outline isinya mau apa saja..



Saya membuat seperti ini (watermark/watermark/...)

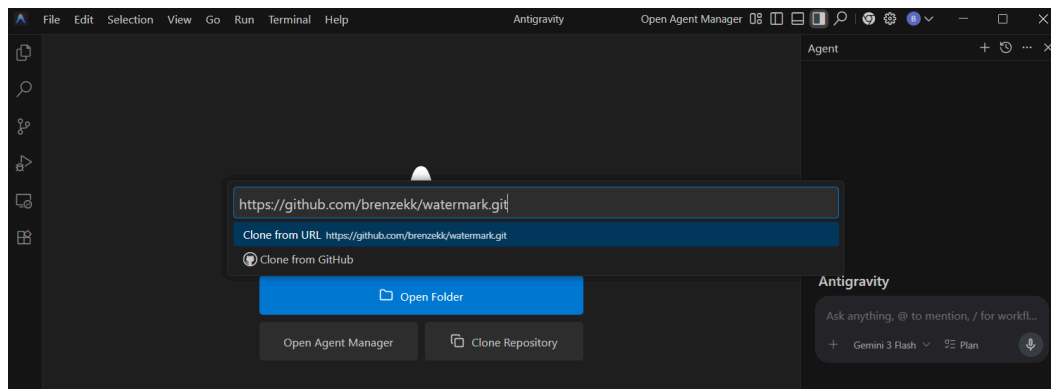
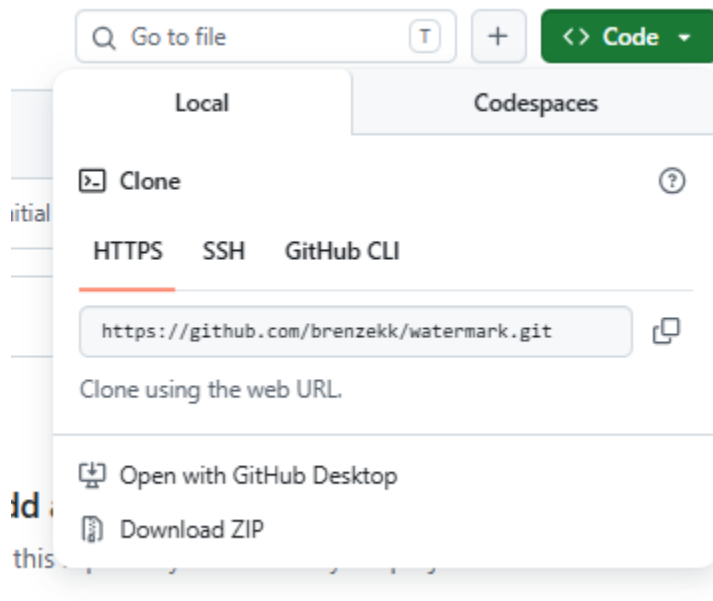


karena sebagai bentuk isolasi *environment* untuk memisahkan antara direktori root dengan direktori *source code*. Direktori terluar untuk `.venv`, metadata, dan lain sebagainya. Sementara yang di dalamnya ada [main.py](#), `assets`, `output`, dan lain sebagainya.

- file [main.py](#) berisi kode program utama yang menjalankan proses watermark
- folder `assets` menyimpan file gambar selfie
- `output` sebagai penampung seluruh hasil eksekusi visual dari setiap sub-proses

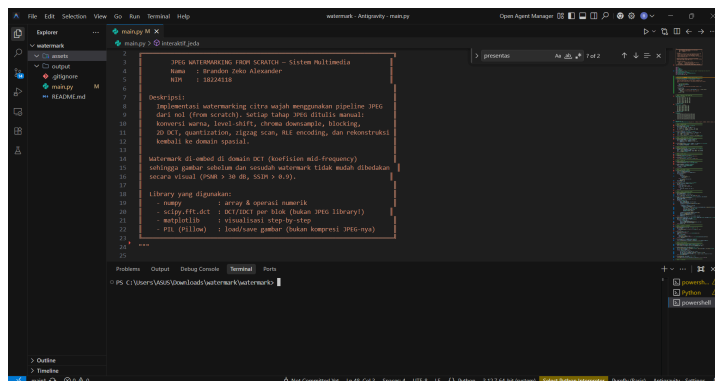
2. Lakukan clone git repository

Setelah membuat strukturnya di folder watermark yang repository maka lakukan clone



Saya clone ke antigravity karena akhir akhir ini dari tubes tubes lainnya terutama teknologi platform memudahkan saya ketika memodifikasi code.

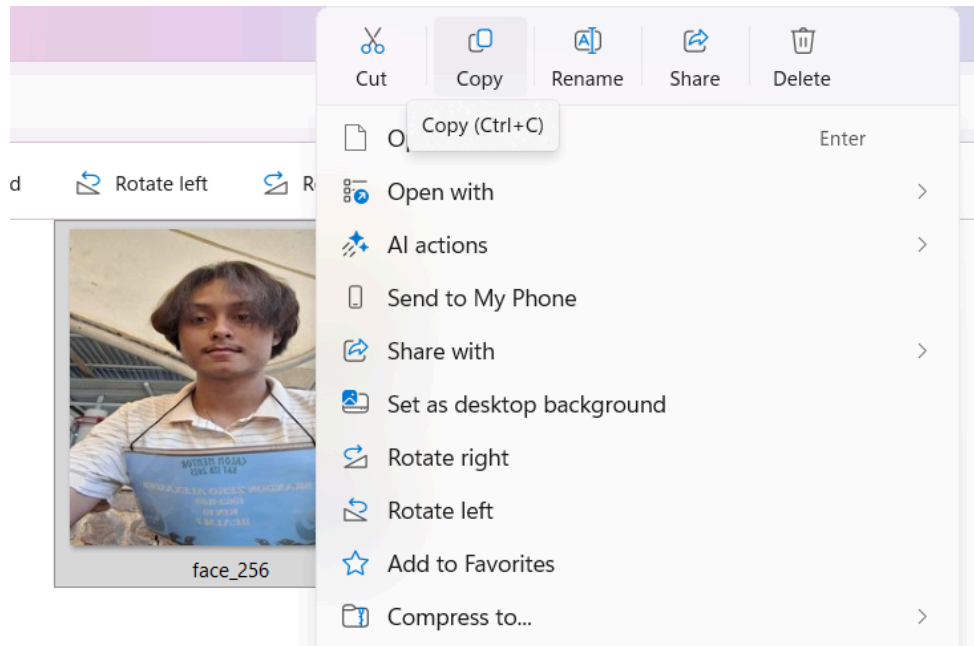
Setelah enter, muncul seperti ini:



Untuk membuat code dari scratch yang akan diisi dalam file [main.py](#) saya meminta claude untuk membuatnya dengan saya memberinya syarat tidak memakai library apapun untuk proses JPEG Compressionnya. Saya hanya menggunakan library untuk menggambar diagram visualisasi step-bystepnya yang akan muncul di output bukan untuk melakukan kompresi JPEG-nya sehingga aman.

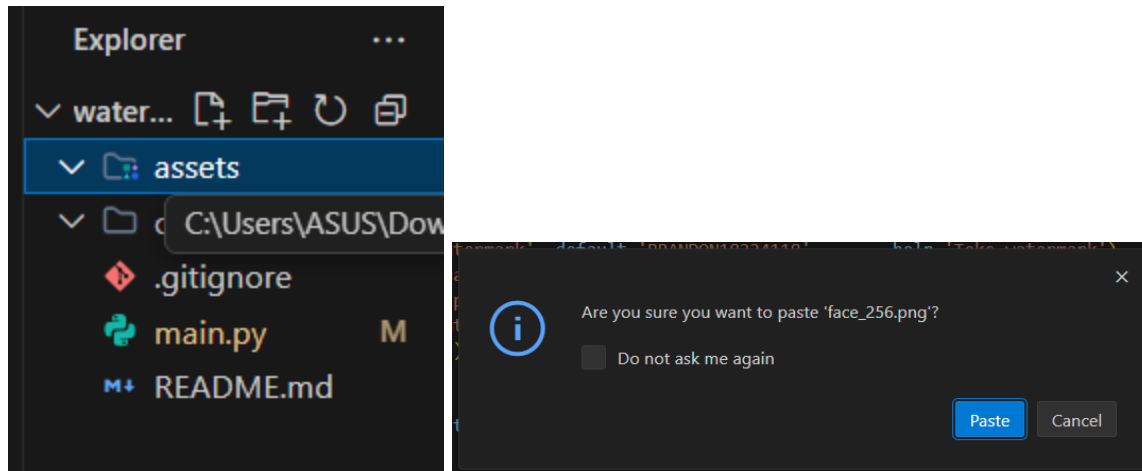
3. Masukkan file foto selfie di folder “assets”

Copy file selfie dari file explorer kalian yang mau diproses ke folder “assets”

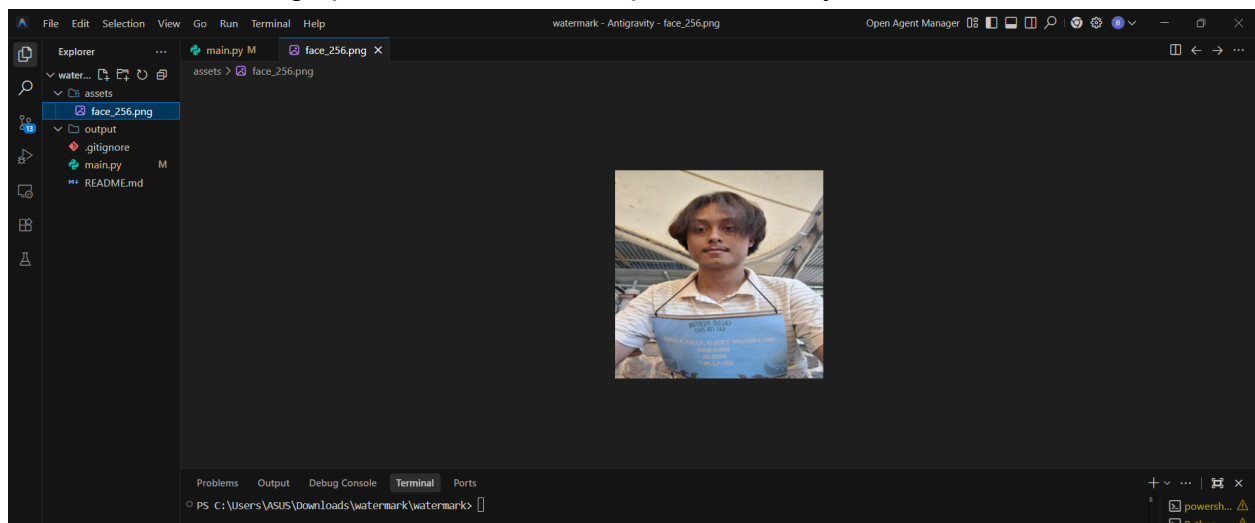


kalau bisa gambarnya 256 x 256 agar saat DCT menjadi 8x8 piksel akan terkesktrak secara sempurna menjadi 1.024 blok independen (32 x 32 blok) tanpa menghasilkan sisa matriks (*remainder*).

Klik ctrl + v di sini dan paste



Sudah berhasil menginput file selfie untuk diproses nantinya



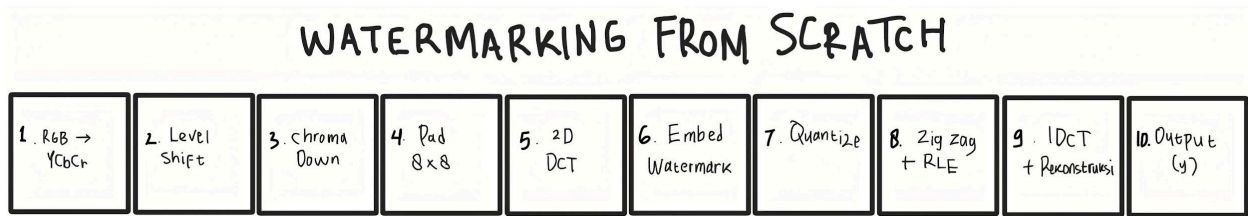
Lalu dilakukan penyesuaian *path directory* pada fungsi *argument parser* dengan mengisi yang saya select (berwarna biru) ke nama file selfie yang diupload tadi pada assets.

```

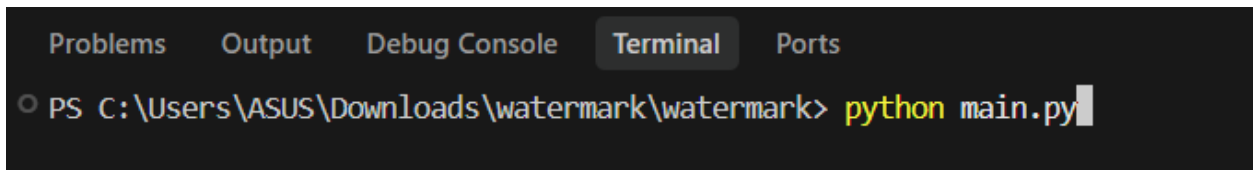
350 # -----
351 # MAIN PROGRAM
352 # -----
353 def main():
354     parser = argparse.ArgumentParser()
355     parser.add_argument('--image', default='assets/face_256.png', help='Path ke gambar input')
356     parser.add_argument('--watermark', default='BRANDON18224118', help='Teks watermark')
357     parser.add_argument('--quality', type=int, default=75, help='Quality Factor')
358     parser.add_argument('--alpha', type=float, default=30.0, help='Alpha Embedding')
359     parser.add_argument('--output', default='output', help='Folder Output')
360     args = parser.parse_args()

```

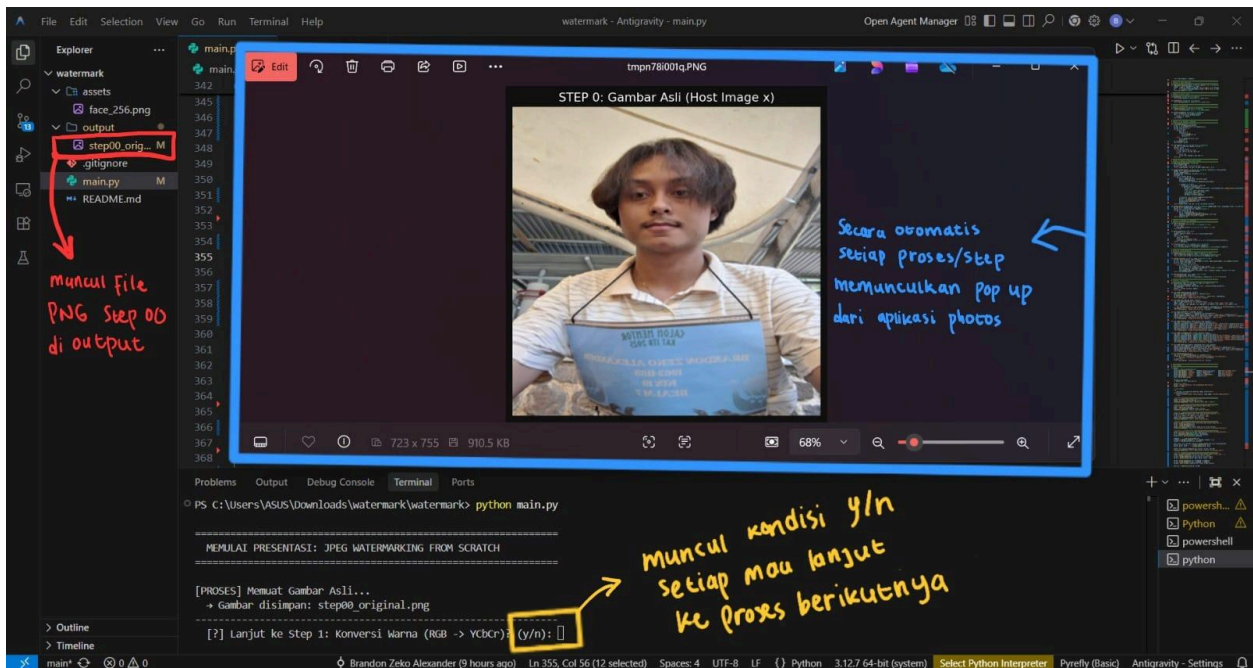
4. Menjalankan Program & step 00: Inisialisasi *host image*



Ketik pada terminal python [main.py](#)



Nah akan muncul seperti ini:



Prosesnya dimulai dari program akan mencari gambar di dalam folder assets/ dengan nama `face_256.png`

```
parser.add_argument('--image', default='assets/face_256.png', help='Path ke gambar input')
parser.add_argument('--watermark', default='BRANDON18224118', help='Teks watermark')
parser.add_argument('--quality', type=int, default=75, help='Quality Factor')
parser.add_argument('--alpha', type=float, default=30.0, help='Alpha Embedding')
parser.add_argument('--output', default='output', help='Folder Output')
args = parser.parse_args()

OUT_DIR = args.output
os.makedirs(OUT_DIR, exist_ok=True)
```

Di sini menggunakan library Pillow (PIL) untuk buka file face_256.png, ada .convert('RGB') buat cek RGB & gaada warna transparan, lalu np.array mengubah foto menjadi jutaan angka matriks (ukuran 256x256x3) disimpan ke variabel img_rgb.

```
# [LOAD GAMBAR]
try:
    img_rgb = np.array(Image.open(args.image).convert('RGB'))
except Exception as e:
    print(f"\n[ERROR] Gambar tidak ditemukan: {args.image}. Pastikan namanya benar!")
    return
```

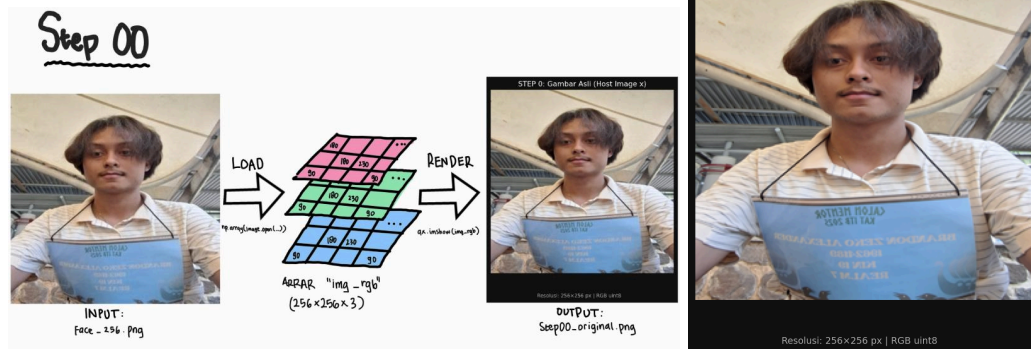
Di sini dilakukan pemanggilan fungsi vis_step0_original, dalam fungsi tersebut program memvisualisasikan ulang angka-angka dari variabel img_rgb tadi,

```
def vis_step0_original(img_rgb, out_dir):
    fig, ax = plt.subplots(figsize=(6, 6), facecolor='#111')
    ax.imshow(img_rgb); ax.axis('off')
    ax.set_title('STEP 0: Gambar Asli (Host Image x)', color='white', fontsize=13)
    return save_fig(fig, 'step00_original.png', out_dir)
```

^
|
(memanggil fungsi atas)
|
|

```
# — STEP 0 —
print("\n[PROSES] Memuat Gambar Asli...")
path0 = vis_step0_original(img_rgb, OUT_DIR)
interaktif_jeda(path0, "Step 1: Konversi Warna (RGB -> YCbCr)")
```

lalu akan tersimpan hasilnya dengan nama step00_original.png melalui plt.savefig hingga muncul ini:



Bersamaan dengan itu, terdapat code yang memanfaatkan pemanggilan tingkat OS yang membuat muncul pop up si gambar step00_original.png.

Saya ingin setiap proses menunjukkan hasil/kondisi yang dilakukan terhadap gambar selfi yang diinput dan setiap selesai menjalani suatu proses/step akan ada kondisi y/n supaya berlangsung secara satu per satu dan terlihat **apa yang baru saja dilakukan**.

```

39 # -----
40 # FUNGSI MODE INTERAKTIF (POP-UP & JEDA)
41 # -----
42 def interaktif_jeda(image_path: str, next_step_name: str = None):
43     """
44     Fungsi untuk memunculkan gambar ke layar (pop-up) menggunakan viewer
45     bawaan OS (seperti Windows Photos), lalu meminta persetujuan Y/N di terminal.
46     """
47     try: → Tab to Jump
48         # Munculkan pop-up gambar
49         img = Image.open(image_path)
50         img.show()
51     except Exception as e:
52         print(f" [!] Gagal memunculkan pop-up gambar otomatis: {e}")
53
54     if next_step_name:
55         print("-" * 62)
56         jawab = input(f" [?] Lanjut ke {next_step_name}? (y/n): ").strip().lower()
57         if jawab == 'n':
58             print("\n [!] Proses dihentikan. Sampai jumpa!\n")
59             sys.exit(0)
60         print("-" * 62)

```

[?] Lanjut ke Step 1: Konversi Warna (RGB -> YCbCr)? (y/n):

5. Step 01: RGB → YCbCr

Ketik “y” untuk masuk ke step 01 konversi RGB ke YCbCr

```
Problems Output Debug Console Terminal Ports
O =====
  MEMULAI PRESENTASI: JPEG WATERMARKING FROM SCRATCH
  =====

  [PROSES] Memuat Gambar Asli...
  → Gambar disimpan: step00_original.png
  -----
  [?] Lanjut ke Step 1: Konversi Warna (RGB -> YCbCr)? (y/n): y
```

Program selanjutnya akan melanjutkan ke step selanjutnya, yaitu step 01 Konversi warna RGB -> YCbCr, terkeksekusilah ini:

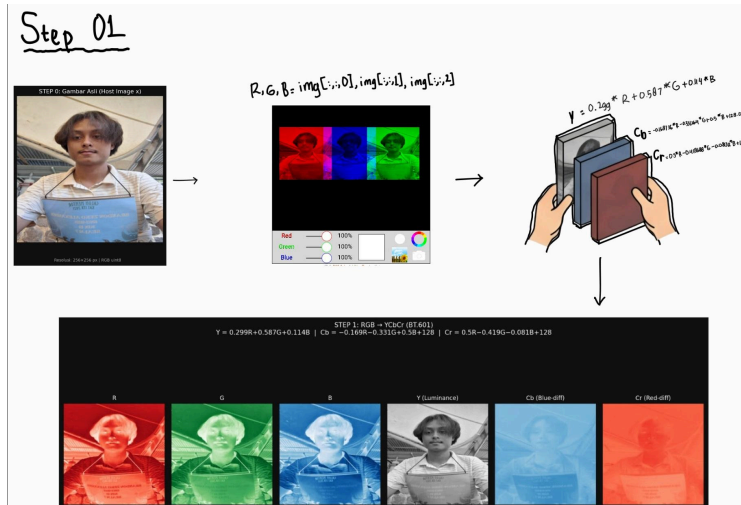
```
# — STEP 1 —
print("\n[PROSES] Menghitung rumus konversi RGB ke YCbCr...")
ycbcr = rgb_to_ycbcr(img_rgb)
path1 = vis_step1_ycbcr(img_rgb, ycbcr, OUT_DIR)
interaktif_jeda(path1, "Step 2: Level Shifting (-128)")
```

Memanggil fungsi `rgb_to_ycbcr()`, di bawah:

```
# =====
# BAGIAN 2 — KONVERSI WARNA
# =====
def rgb_to_ycbcr(img_rgb: np.ndarray) -> np.ndarray:
    img = img_rgb.astype(np.float64)
    R, G, B = img[:, :, 0], img[:, :, 1], img[:, :, 2]
    Y = 0.299 * R + 0.587 * G + 0.114 * B
    Cb = -0.168736 * R - 0.331264 * G + 0.5 * B + 128.0
    Cr = 0.5 * R - 0.418688 * G - 0.081312 * B + 128.0
    return np.stack([Y, Cb, Cr], axis=2)

def ycbcr_to_rgb(img_ycbcr: np.ndarray) -> np.ndarray:
    Y, Cb, Cr = img_ycbcr[:, :, 0], img_ycbcr[:, :, 1], img_ycbcr[:, :, 2]
    R = Y + 1.402 * (Cr - 128.0)
    G = Y - 0.344136 * (Cb - 128.0) - 0.714136 * (Cr - 128.0)
    B = Y + 1.772 * (Cb - 128.0)
    return np.clip(np.stack([R, G, B], axis=2), 0, 255).astype(np.uint8)
```

Cara kerjanya,



- Fungsi `rgb_to_ycbcr()`

Fungsi ini tuh ngambil data matriks warna dari foto selfie saya (`img_rgb`) yang tadi ada di step 00 lalu memisahkan elemen pencahayaan Y (Luminance) dari elemen warna benerannya hingga tersisa Cb & Cr (Chrominance).

terdapat beberapa detail yang cukup kompleks

`img = img_rgb.astype(np.float64)`

Intinya dia ingin mengubah angka piksel di gambar asli saya (bil. bulat) menjadi desimal (`float64`) supaya tidak terjadi kerusakan (dibulatkan paksa oleh komputer) pada gradasi warna saat dikalikan dengan pecahan desimal (0.299 atau -0.168736).

`R, G, B = img[:, :, 0], img[:, :, 1], img[:, :, 2]`

Dikuliti dari 3D menjadi lembar matriks 2D yang masing-masingnya mengambil semua piksel berdasarkan salurannya (R, G, B)

`Y = 0.299 * R + 0.587 * G + 0.114 * B`

`Cb = -0.168736 * R - 0.331264 * G + 0.5 * B + 128.0`

`Cr = 0.5 * R - 0.418688 * G - 0.081312 * B + 128.0`

Semua ini tuh rumus yang sudah ada,

Untuk yang Y adalah hasil penggabungan R,G,B menjadi foto hitam-putih,

Untuk yang Cb&Cr terdapat +128.0 karena dilakukan *shifting*/digeser ke atas 128 kali agar tidak bernilai minus.

`return np.stack([Y, Cb, Cr], axis=2)`

Tiga lembar tadi ditumpuk kembali menjadi matriks 3D.

- Fungsi `ycbcr_to_rgb()`

Digunakan agar bisa dikembalikan gambar tersebut menjadi file photo biasa (RGB) yg bisa dibuka di device kita.

6. Step 02: Level Shifting

Klik “y” lagi untuk masuk ke step 02 ini

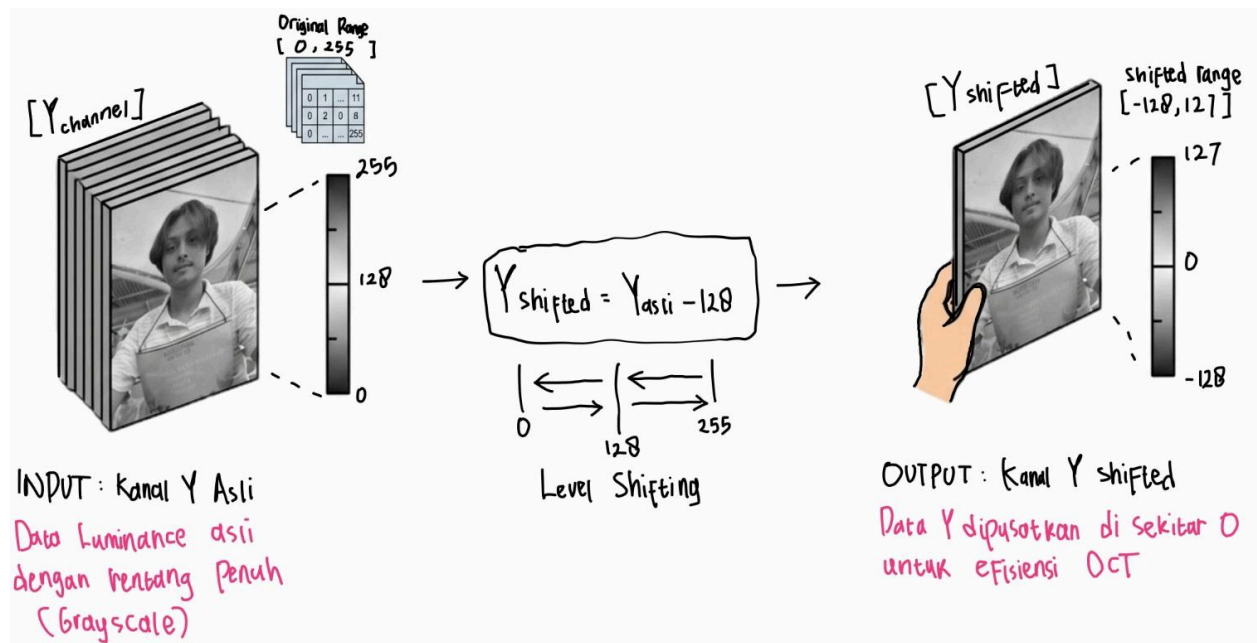
```
[PROSES] Menghitung rumus konversi RGB ke YCbCr...
```

```
→ Gambar disimpan: step01_rgb_to_ycbcr.png
```

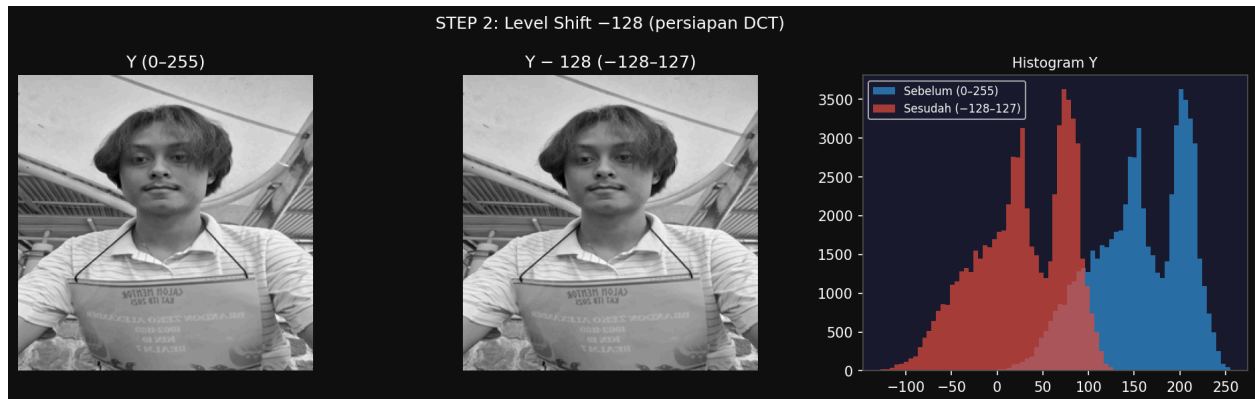
```
-----  
[?] Lanjut ke Step 2: Level Shifting (-128)? (y/n): y
```

Program selanjutnya akan melanjutkan ke step berikutnya, yaitu step 02 Level Shifting, tereksekusilah ini:

```
# — STEP 2 —  
print("\n[PROSES] Melakukan Level Shift untuk persiapan nilai DCT...")  
Y, Cb, Cr = ycbcr[:, :, 0], ycbcr[:, :, 1], ycbcr[:, :, 2]  
Y_sh, Cb_sh, Cr_sh = Y - 128.0, Cb - 128.0, Cr - 128.0  
path2 = vis_step2_levelshift(Y, Y_sh, OUT_DIR)  
interaktif_jeda(path2, "Step 3: Chroma Downsampling")
```



Tidak ada fungsi yang memanggil proses kerja step 2 di sini, hanya memanggil fungsi visualisasi (`vis_step2_levelshift()`) dan fungsi `interaktif_jeda()` untuk sama-sama bisa memunculkan pop up gambar ini:



Y, Cb, Cr = ycbcr[:, :, 0], ycbcr[:, :, 1], ycbcr[:, :, 2]

Proses level shift step 02 ini dimulai dengan melakukan pemotongan/*slicing* pada array ycbcr untuk memisahkan ketiga salurannya (Y, Cb, Cr).

Y_sh, Cb_sh, Cr_sh = Y - 128.0, Cb - 128.0, Cr - 128.0

Di sinilah dilakukan shift yang bertujuan agar nilai input yang dipusatkan di sekitar nol agar prapemrosesan DCT lebih efektif. Kenapa bisa efektif? Karena ini akan membuat pengecilan angka rata-rata pada blok gambar (koefisien DC) secara drastis. Dengan angka yang menjadi jauh lebih kecil, proses kompresi selanjutnya akan membutuhkan memori (bit) yang jauh lebih sedikit untuk menyimpannya.

7. Step 03: Chroma Downsampling

Ketik "y" untuk masuk ke step 03

```
[PROSES] Melakukan Level Shift untuk persiapan nilai DCT...
→ Gambar disimpan: step02_level_shift.png
-----
[?] Lanjut ke Step 3: Chroma Downsampling? (y/n): y
```

Program selanjutnya akan melanjutkan ke step berikutnya, yaitu step 03 Chroma Downsampling, tereksekusilah ini:

```
# — STEP 3 —
print("\n[PROSES] Membuang detail warna yang tidak sensitif bagi mata...")
Cb_dn = chroma_downsample(Cb_sh)
Cr_dn = chroma_downsample(Cr_sh)
path3 = vis_step3_chroma(Y, Cb_sh, Cb_dn, Cr_sh, Cr_dn, OUT_DIR)
interaktif_jeda(path3, "Step 4 & 5: Blocking, Proses DCT & Penyisipan Watermark")
```

Di sini tujuan prosesnya kan mau mengurangi detail warna karena mata manusia lebih peka ke cahaya dibanding warna dengan prinsip kompresi 4:2:0. Sehingga dimasukkanlah saluran warna biru dan merah yang hasil shift tadi ke variabel Cr_dn & Cb_dn untuk nantinya diproses dalam fungsi chroma_downsample().

Memanggil fungsi `chroma_downsample()`, di bawah:

```
# =====  
# BAGIAN 3 — CHROMA DOWNSAMPLING & UPSAMPLING  
# =====  
def chroma_downsample(channel: np.ndarray) -> np.ndarray:  
    H2, W2 = channel.shape[0] // 2, channel.shape[1] // 2  
    out = (channel[0::2, 0::2] + channel[1::2, 0::2] +  
           channel[0::2, 1::2] + channel[1::2, 1::2]) / 4.0  
    return out[:H2, :W2]  
  
def chroma_upsample(channel: np.ndarray, target_H: int, target_W: int) -> np.ndarray:  
    up = np.repeat(np.repeat(channel, 2, axis=0), 2, axis=1)  
    return up[:target_H, :target_W]
```

H2, W2 = channel.shape[0] // 2, channel.shape[1] // 2

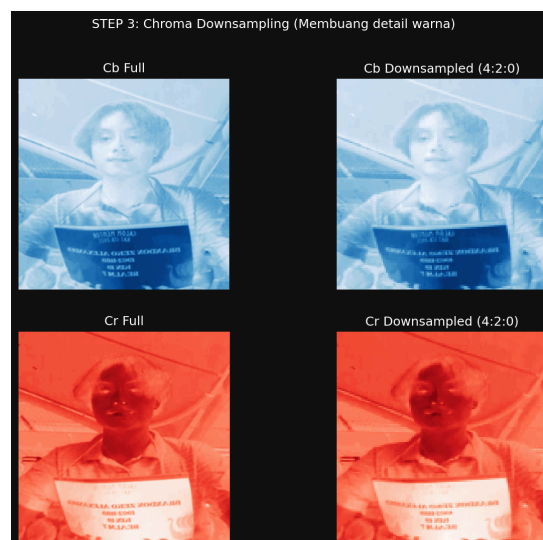
Program mengambil tinggi dan lebar matriks warna aslinya (256x256) lalu membaginya dengan 2 sehingga target ukuran barunya sekarang adalah 128x128.

**out = (channel[0::2, 0::2] + channel[1::2, 0::2] +
 channel[0::2, 1::2] + channel[1::2, 1::2]) / 4.0**

Membagi gambar menjadi kotak-kotak kecil berukuran 2x2 piksel (total 4 piksel). Kode di atas mengambil nilai dari piksel Kiri-Atas, Kiri-Bawah, Kanan-Atas, dan Kanan-Bawah dari setiap kotak, menjumlahkannya, lalu **dibagi 4.0**. Hasilnya adalah 1 piksel baru yang merupakan nilai rata-rata dari 4 piksel lama.

return out[:H2, :W2]

Program mengembalikan matriks warna yang baru. Ukurannya sekarang sudah menyusut drastis menjadi seperempat dari data aslinya (128x128), yang menyebabkan warnanya terlihat lebih buram/kotak-kotak



8. Step 04-07: Blocking, Proses DCT & Penyisipan Watermark

Ketik "y" untuk lanjut ke step 04 Blocking, Proses DCT & Penyisipan Watermark

```
[PROSES] Membuang detail warna yang tidak sensitif bagi mata...  
→ Gambar disimpan: step03_chroma_downsample.png
```

```
[?] Lanjut ke Step 4 & 5: Blocking, Proses DCT & Penyisipan Watermark? (y/n): y
```

Program selanjutnya akan melanjutkan ke step berikutnya, yaitu step berikut, tereksekusilah ini:

```
# — STEP 4-7 (PIPELINE UTAMA) —  
print("\n[PROSES] Memotong blok 8x8, menghitung matriks DCT, dan Embed Watermark...")  
qt_Y = make_quant_table(LUMA_QT_BASE, args.quality)  
qt_C = make_quant_table(CHROMA_QT_BASE, args.quality)  
wm_bits_list = text_to_bits(args.watermark)  
  
padded_Y, _, _ = pad_to_multiple8(Y_sh)  
n_blocks = (padded_Y.shape[0] // 8) * (padded_Y.shape[1] // 8)  
wm_bits = tile_bits(wm_bits_list, n_blocks)  
  
enc_Y, ps_Y, os_Y, dct_bef, dct_aft = encode_channel(Y_sh, qt_Y, wm_bits=wm_bits)  
enc_Cb, ps_Cb, os_Cb, _, _ = encode_channel(Cb_dn, qt_C)  
enc_Cr, ps_Cr, os_Cr, _, _ = encode_channel(Cr_dn, qt_C)  
  
path4 = vis_step4_dct_embed(dct_bef, dct_aft, wm_bits, OUT_DIR)  
interaktif_jeda(path4, "Step 8: Rekonstruksi & Analisis Perbandingan Akhir")
```

make_quant_table(...)

Membuat dua jenis matriks penentu seberapa kuat gambar akan dikompresi (satu untuk luma/Y, dan yang C untuk chroma). Matriks disesuaikan dengan nilai Quality Factor yang kita atur.

text_to_bits(...) & tile_bits(...)

Teks *watermark* diubah menjadi angka biner murni (0 dan 1). Karena jumlah blok pada gambar sangat banyak, rentetan bit ini digandakan/diulang (tile_bits) agar setiap blok gambar kebagian satu bit untuk disisipkan.

pad_to_multiple8(...)

Rumus matematis DCT ngewajibin gambar berukuran blok 8x8 piksel. Fungsi ini "menambal" pinggiran gambar dengan piksel palsu (jika perlu) agar panjang dan lebarnya habis dibagi 8.

encode_channel(...)

Ini adalah instruksi eksekusi utama. Alih-alih menulis ratusan baris kode untuk rumus DCT, Kuantisasi, dan Zigzag, semuanya dibungkus rapi ke dalam satu fungsi ini.

Pemanggilan Pertama (enc_Y)

Fungsi ini memproses saluran kecerahan (Y). Perhatikan bahwa ada parameter tambahan `wm_bits=wm_bits`. Ini artinya, **khusus pada saluran kecerahan inilah watermark disisipkan**.

Pemanggilan Kedua & Ketiga (enc_Cb, enc_Cr)

Fungsi dipanggil lagi untuk saluran warna (Biru dan Merah). Di sini, fungsi tidak menyertakan parameter `wm_bits`. Artinya, saluran warna hanya dikompresi saja menggunakan aturan standar JPEG tanpa disisipi pesan apa pun, agar warna asli wajah tidak berubah drastis/rusak.

Fungsi diambil dari bagian-bagian berikut:

```
# =====
# BAGIAN 4 – TABEL KUANTISASI
# =====
def make_quant_table(base_table: np.ndarray, quality: int) -> np.ndarray:
    quality = int(np.clip(quality, 1, 100))
    scale = 5000.0 / quality if quality < 50 else 200.0 - 2.0 * quality
    qt = np.clip(np.floor(base_table * scale / 100.0 + 0.5), 1, 255)
    return qt.astype(np.float64)
```

```
# =====
# BAGIAN 8 – PADDING & ENCODE/DECODE CHANNEL
# =====
def pad_to_multiple8(channel: np.ndarray):
    H, W = channel.shape
    pH, pW = (8 - H % 8) % 8, (8 - W % 8) % 8
    return np.pad(channel, ((0, pH), (0, pW)), mode='edge'), H, W

def encode_channel(channel: np.ndarray, qt: np.ndarray, wm_bits=None, start_block=0):
    padded, H, W = pad_to_multiple8(channel)
    pH, pW = padded.shape
    encoded_blocks, dct_before, dct_after = [], [], []
    block_idx = 0
    for row in range(0, pH, 8):
        for col in range(0, pW, 8):
            block = padded[row:row+8, col:col+8].copy()
            dct_block = dct2d(block)
            if block_idx < 16: dct_before.append(dct_block.copy())

            if wm_bits is not None:
                global_idx = start_block + block_idx
                if global_idx < len(wm_bits):
                    zz, bit, coeff = zigzag_scan(dct_block), wm_bits[global_idx], zigzag_scan(dct_block)[EMBED_POSITION]
                    ALPHA = 30.0
                    if bit == 1: coeff = max(coeff, ALPHA)
                    else: coeff = min(coeff, -ALPHA)
                    zz[EMBED_POSITION] = coeff
                    dct_block = inverse_zigzag(zz)

            if block_idx < 16: dct_after.append(dct_block.copy())
            q_block = np.round(dct_block / qt)
            dc, ac = rle_encode(zigzag_scan(q_block))
            encoded_blocks.append((dc, ac))
            block_idx += 1
    return encoded_blocks, (pH, pW), (H, W), dct_before, dct_after
```

```
def decode_channel(encoded_blocks: list, qt: np.ndarray, padded_shape: tuple, orig_shape: tuple) -> np.ndarray:
    pH, pW, H, W = *padded_shape, *orig_shape
    canvas, idx = np.zeros((pH, pW), dtype=np.float64), 0
    for row in range(0, pH, 8):
        for col in range(0, pW, 8):
            dc, ac = encoded_blocks[idx]; idx += 1
            spatial = idct2d(inverse_zigzag(rle_decode(dc, ac)) * qt)
            canvas[row:row+8, col:col+8] = spatial
    return canvas[:H, :W]
```

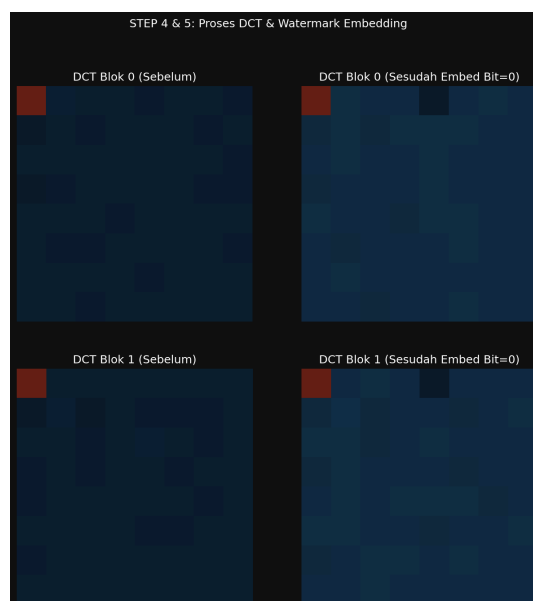
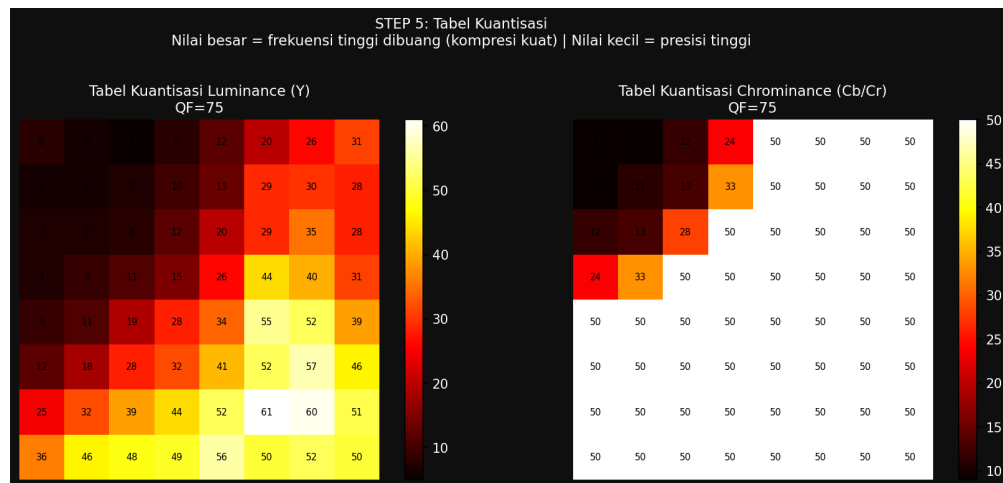
```
# =====
# BAGIAN 9 - WATERMARK BIT CONVERSION
# =====

def text_to_bits(text: str) -> list:
    bits = [(len(text) >> i) & 1 for i in range(15, -1, -1)]
    for ch in text:
        bits.extend([(ord(ch) >> i) & 1 for i in range(7, -1, -1)])
    return bits

def bits_to_text(bits: list) -> str:
    if len(bits) < 16: return ""
    length = sum((b << (15-i)) for i, b in enumerate(bits[:16]))
    chars = []
    for i in range(length):
        start = 16 + i * 8
        if start + 8 > len(bits): break
        chars.append(chr(sum((b << (7-j)) for j, b in enumerate(bits[start:start+8]))))
    return ''.join(chars)

def tile_bits(wm_bits: list, n_blocks: int) -> np.ndarray:
    if not wm_bits: return np.zeros(n_blocks, dtype=int)
    return np.array((wm_bits * ((n_blocks // len(wm_bits)) + 1))[:n_blocks], dtype=int)
```

Ilustrasi untuk step ini seperti ini:



9. Step 8-10: Rekonstruksi & Analisis Perbandingan Akhir

Tekan “y” untuk kumpulan step terakhir

```
[PROSES] Memotong blok 8x8, menghitung matriks DCT, dan Embed Watermark...
→ Gambar disimpan: step04_blocks_dct_embed.png
-----
[?] Lanjut ke Step 8 - 10: Rekonstruksi & Analisis Perbandingan Akhir? (y/n): y
```

Program untuk step ini akan mengeksekusi ini:

```
# — STEP 8-last —
print("\n[PROSES] Mengembalikan matriks angka menjadi gambar visual (IDCT)...")
Y_rec = decode_channel(enc_Y, qt_Y, ps_Y, os_Y) + 128.0
Cb_rec = decode_channel(enc_Cb, qt_C, ps_Cb, os_Cb) + 128.0
Cr_rec = decode_channel(enc_Cr, qt_C, ps_Cr, os_Cr) + 128.0

Cb_up = chroma_upsample(Cb_rec, Y.shape[0], Y.shape[1])
Cr_up = chroma_upsample(Cr_rec, Y.shape[0], Y.shape[1])
wm_rgb = ycbcr_to_rgb(np.stack([Y_rec, Cb_up, Cr_up], axis=2))

psnr_val = compute_psnr(img_rgb, wm_rgb)
ssim_val = compute_ssim(img_rgb, wm_rgb)

path8 = vis_step8_comparison(img_rgb, wm_rgb, psnr_val, ssim_val, OUT_DIR)

print("\n[HASIL AKHIR]")
print(f"  PSNR: {psnr_val:.2f} dB")
print(f"  SSIM: {ssim_val:.4f}")

interaktif_jeda(path8, None) # None berarti ini langkah terakhir

print(f"\n{'='*62}")
print("  PROSES SELESAI DENGAN SUKSES!")
print("  Semua gambar bukti telah diamankan di folder 'output/'.")
print(f"\n{'='*62}")
```

decode_channel(...)

Data saluran Y, Cb, dan Cr yang sudah dikompresi (dalam wujud array 1D hasil RLE) dimasukkan ke fungsi ini. Fungsi ini akan mekar-kan data (*RLE Decode*), menyusunnya kembali ke kotak 8x8 (*Inverse Zigzag*), mengalikan kembali dengan tabel kuantisasi (*Dequantization*), dan puncaknya menggunakan rumus **IDCT (Inverse Discrete Cosine Transform)** untuk merubah data frekuensi kembali menjadi nilai kecerahan piksel.

+ 128.0

INGAT LAGI DI STEP 02 LIFTING? Di sana kan ada mengurangi nilai piksel dengan 128. Nah, di sini kita mengembalikannya (ditambah 128) agar rentang warnanya kembali normal ke 0-255.

chroma_upsample(...)

Pada Step 03, saluran warna (Cb dan Cr) diperkecil jadi setengahnya. Fungsi ini membesarkan kembali matriks tersebut ke ukuran aslinya (256x256) dengan cara menduplikasi/menggandakan piksel yang ada agar ukurannya sejajar dengan saluran Y.

np.stack(...) & ycbcr_to_rgb(...)

Ketiga matriks (Y, Cb, Cr) yang tadinya terpisah, kini ditumpuk (stack) menjadi satu kesatuan gambar 3D. Setelah itu, format YCbCr dikonversi kembali menjadi RGB (Red, Green, Blue) karena layar monitor kita hanya bisa menampilkan warna dalam format RGB.

Untuk Quality Assesment

Gambar asli (**img_rgb**) dan gambar hasil yang sudah mengandung watermark (**wm_rgb**) diadu. Program menghitung **PSNR** (Peak Signal-to-Noise Ratio) dan **SSIM** (Structural Similarity Index) untuk melihat seberapa hancur gambar tersebut setelah diproses. Semakin tinggi nilainya (PSNR > 30 dB dan SSIM mendekati 1.0), berarti gambar hasil rekonstruksi terlihat semakin identik dengan gambar asli.

Fungsi diambil dari bagian-bagian berikut:

```
def decode_channel(encoded_blocks: list, qt: np.ndarray, padded_shape: tuple, orig_shape: tuple) -> np.ndarray:
    pH, pW, H, W = *padded_shape, *orig_shape
    canvas, idx = np.zeros((pH, pW), dtype=np.float64), 0
    for row in range(0, pH, 8):
        for col in range(0, pW, 8):
            dc, ac = encoded_blocks[idx]; idx += 1
            spatial = idct2d(inverse_zigzag(rle_decode(dc, ac)) * qt)
            canvas[row:row+8, col:col+8] = spatial
    return canvas[:H, :W]
```

```
#
# BAGIAN 7 - RLE ENCODING / DECODING
#
def rle_encode(zigzag_vec: np.ndarray):
    dc_val = int(round(zigzag_vec[0]))
    ac_vals = [int(round(v)) for v in zigzag_vec[1:]]
    ac_rle, zero_run = [], 0
    for val in ac_vals:
        if val == 0:
            zero_run += 1
            if zero_run == 16:
                ac_rle.append((15, 0))
                zero_run = 0
        else:
            ac_rle.append((zero_run, val))
            zero_run = 0
    ac_rle.append((0, 0))
    return dc_val, ac_rle

def rle_decode(dc_val: int, ac_rle: list) -> np.ndarray:
    vec, idx = np.zeros(64, dtype=np.float64), 1
    vec[0] = dc_val
    for run, val in ac_rle:
        if (run, val) == (0, 0): break
        if (run, val) == (15, 0): idx += 16
        else:
            idx += run
            if idx < 64: vec[idx] = val; idx += 1
    return vec
```

```
# =====
# BAGIAN 6 - ZIGZAG SCAN
# =====
def zigzag_scan(block: np.ndarray) -> np.ndarray:
    return np.array([block[r, c] for r, c in ZIGZAG_ORDER])

def inverse_zigzag(vec: np.ndarray) -> np.ndarray:
    block = np.zeros((8, 8), dtype=np.float64)
    for i, (r, c) in enumerate(ZIGZAG_ORDER):
        block[r, c] = vec[i]
    return block
```

```
# =====
# BAGIAN 5 - 2D DCT / IDCT
# =====
def dct2d(block: np.ndarray) -> np.ndarray:
    return dct(dct(block.T, norm='ortho').T, norm='ortho')

def idct2d(block: np.ndarray) -> np.ndarray:
    return idct(idct(block.T, norm='ortho').T, norm='ortho')
```

```
def chroma_upsample(channel: np.ndarray, target_H: int, target_W: int) -> np.ndarray:
    up = np.repeat(np.repeat(channel, 2, axis=0), 2, axis=1)
    return up[:target_H, :target_W]
```

```
def ycbcr_to_rgb(img_ycbcr: np.ndarray) -> np.ndarray:
    Y, Cb, Cr = img_ycbcr[:, :, 0], img_ycbcr[:, :, 1], img_ycbcr[:, :, 2]
    R = Y + 1.402 * (Cr - 128.0)
    G = Y - 0.344136 * (Cb - 128.0) - 0.714136 * (Cr - 128.0)
    B = Y + 1.772 * (Cb - 128.0)
    return np.clip(np.stack([R, G, B], axis=2), 0, 255).astype(np.uint8)
```

```
# =====
# BAGIAN 10 - METRIK KUALITAS
# =====
def compute_psnr(orig: np.ndarray, recon: np.ndarray) -> float:
    mse = np.mean((orig.astype(np.float64) - recon.astype(np.float64)) ** 2)
    return float('inf') if mse == 0 else 10.0 * np.log10(255.0**2 / mse)

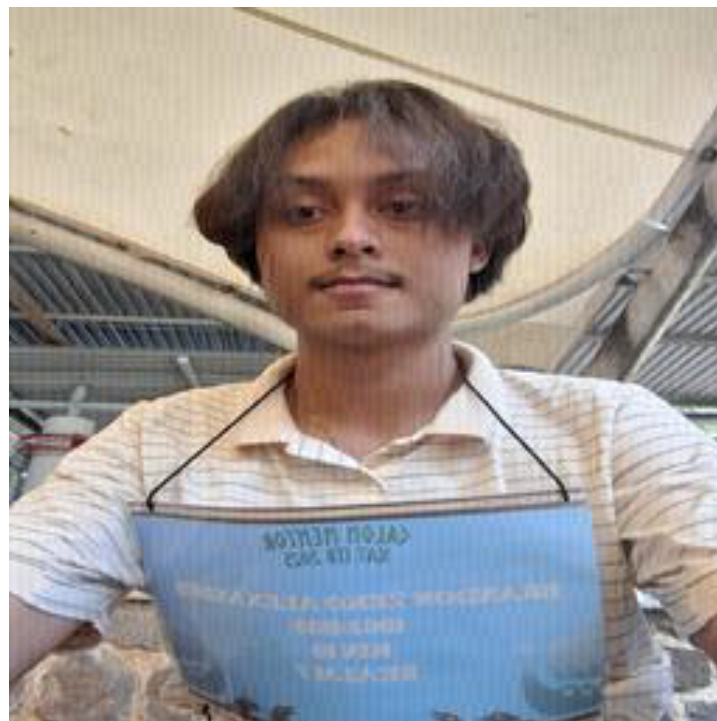
def compute_ssim(img1: np.ndarray, img2: np.ndarray) -> float:
    C1, C2 = (0.01 * 255)**2, (0.03 * 255)**2
    y1, y2 = rgb_to_ycbcr(img1[:, :, 0]).astype(np.float64), rgb_to_ycbcr(img2[:, :, 0]).astype(np.float64)
    H, W, ws, vals = *y1.shape, 8, []
    for r in range(0, H - ws + 1, ws):
        for c in range(0, W - ws + 1, ws):
            p1, p2 = y1[r:r+ws, c:c+ws], y2[r:r+ws, c:c+ws]
            mu1, mu2, s1, s2 = np.mean(p1), np.mean(p2), np.var(p1), np.var(p2)
            s12 = np.mean((p1 - mu1)*(p2 - mu2))
            vals.append(((2*mu1*mu2 + C1)*(2*s12 + C2)) / ((mu1**2 + mu2**2 + C1)*(s1 + s2 + C2)))
    return float(np.mean(vals))

def bit_accuracy(b_orig: list, b_extr: list) -> float:
    n = min(len(b_orig), len(b_extr))
    return sum(a == b for a, b in zip(b_orig[:n], b_extr[:n])) / n if n > 0 else 0.0
```

The diagram illustrates the proposed watermarking scheme through several stages:

- BPSK DCT Coefficients:** An 8x8 grid of coefficients is shown, with a legend indicating 'BPSK DCT Coefficients (x=0, y=0)'.
- Array 1D setelah Zigzag Scan:** A 1D array of 64 coefficients is shown, with a legend indicating 'Array 1D setelah Zigzag Scan (bunyi = posisi watermark)'.
- DCT Coefficient Plot:** A plot showing the distribution of DCT coefficients, with a legend indicating 'DCT Coefficient Plot (x=0, y=0)'.
- Watermarking Process:** A comparison of the original image (x) and the watermarked image (y) is shown, with a legend indicating 'Watermarking Process (x=0, y=0)'.
- PSNR and SSIM Values:** The PSNR and SSIM values are calculated for the watermarked image, with a legend indicating 'PSNR and SSIM Values (x=0, y=0)'.
- Watermarked Image:** The final watermarked image is shown, with a legend indicating 'Watermarked Image (x=0, y=0)'.

10. Output (Watermarked Face)



Sekian Terimakasih